Dimensionality Reduction with ChatGPT

Centered PCA, Uncentered PCA, Classical MDS, and Minkowski Distances

In this project, you will build reusable AI-assisted workflows for data acquisition, principal component analysis, and classical multidimensional scaling. The project ends with a comparative investigation of Minkowski geometry on three datasets selected by Codex.

Learning objectives

- Use ChatGPT to inspect an unfamiliar data source and identify the observation-feature matrix.
- Distinguish centered PCA from uncentered PCA and verify that selected library routines implement the definitions used in class.
- Translate the classical MDS derivation from the course handout into a correct, testable program.
- Investigate how the Minkowski exponent q changes pairwise geometry and its embedding across several datasets.

Project overview

Stage 1: Data acquisition

Primary tool: ChatGPT.

Your responsibility: Write a reusable prompt that searches for and interprets the most likely UCI dataset matching a short query. Call it `Prompt1.txt`. Execute it by giving ChatGPT one message containing the contents of `Prompt1.txt`, followed by one or more whitespace characters (spaces and/or line breaks), followed by `dataset`.

For the required development run, use the argument `Iris`.

Output: A numeric matrix in CSV format. The file should be named `dataset.csv`; the development run therefore produces `Iris.csv`.

Deliverable: `Prompt1.txt`.

Stage 2: PCA

Primary tool: Codex and Python libraries.

Your responsibility: Write a reusable prompt that selects suitable Python routines for computing uncentered and centered PCA and applies them to a named dataset. The output should be saved in PNG files. Call the prompt `Prompt2.txt`. Execute it by giving Codex one message containing the contents of `Prompt2.txt`, followed by one or more whitespace characters (spaces and/or line breaks), followed by `dataset`. You must separately verify exact agreement with the course definitions; the human grader will check this.

You may assume that the file `dataset.csv` already exists. Use `Iris` for the required development run.

Outputs:

`dataset_upca_2D.png` and `dataset_cpca_2D.png`;
`dataset_upca_3D.png` and `dataset_cpca_3D.png`.

Deliverables: `Prompt2.txt` and the four PNG files generated by the required `Iris` development run.

Stage 3: Classical MDS

Primary tool: Codex.

Your responsibility: Attach the class handout `MDS.pdf` and submit `Prompt3.txt`. The prompt must produce `mds.py` with the command-line interface specified in Part 3, then use it to produce two- and three-dimensional $q = 2$ embeddings of `Iris.csv`.

Outputs: `mds.py`, `dataset_mds_q2_2D.png`, and `dataset_mds_q2_3D.png`.

Deliverables: `Prompt3.txt`, `mds.py`, and the two PNG files `Iris_mds_q2_2D.png` and `Iris_mds_q2_3D.png`.

Stage 4: Investigation

Primary tool: Codex.

Your responsibility: Write a self-contained prompt that instructs Codex to select exactly three eligible UCI datasets, run `mds.py`, and conduct a comparative investigation of q. Call it `Prompt4.txt`. Execute it by giving Codex the contents of `Prompt4.txt`, without a dataset-specific argument. You may assume that `mds.py` already exists.

Output: `q_analysis.md`, with supporting images when they strengthen the analysis.

Deliverables: `Prompt4.txt`, `q_analysis.md`, and any additional files and documentation needed to describe and reproduce the experiments.

Execution sequence

Run the four prompts separately and in order. `Prompt1.txt` creates the shared `dataset.csv` file; `Prompt2.txt` uses that file; `Prompt3.txt` creates the general MDS program and its development-dataset examples; and `Prompt4.txt` uses `mds.py` and selects three datasets of its own. In the notation below, a prompt filename means the contents of that file, not merely its local filename. For `Prompt1.txt` and `Prompt2.txt`, a prompt filename followed by `dataset` means one message to the specified tool consisting of the prompt contents, then one or more whitespace characters (spaces and/or line breaks), then the dataset query. `Prompt4.txt` receives no dataset query.

- `Prompt1.txt dataset` $\longrightarrow$ `dataset.csv`.
- `Prompt2.txt dataset` $\longrightarrow$ centered and uncentered PCA visualizations.
- `Prompt3.txt` plus attached `MDS.pdf` $\longrightarrow$ `mds.py` and the 2D and 3D $q = 2$ development-dataset visualizations.
- `Prompt4.txt` plus `mds.py` $\longrightarrow$ comparative `q_analysis.md`, with supporting figures when useful.

Important: In Stages 1–2, PCA operates directly on `dataset.csv`. Stage 4 instead operates on the three datasets selected by Codex. PCA coordinates are not inputs to MDS; the $q = 2$ comparison is a validation of two independently computed configurations.

Dataset argument and file naming

In Stages 1–2, `dataset` denotes the exact argument supplied after the prompt. It is a short search query and a filename stem, not necessarily the official UCI dataset title. Treat underscores as spaces when searching the UCI repository, and use the underscore-separated words as search keywords. Select the most likely matching UCI dataset without asking the user to disambiguate. Record its official title, UCI dataset ID, URL, and the reason it was selected.

Use the original argument unchanged, including its capitalization and underscores, as the filename stem. For example, `wisconsin_cancer_diagnostic` is searched as "wisconsin cancer diagnostic," finding the UCI dataset *Breast Cancer Wisconsin (Diagnostic)*. It produces the file `wisconsin_cancer_diagnostic.csv`. The required development argument `Iris` produces `Iris.csv` and files such as `Iris_upca_2D.png`.

Rules that apply throughout

- In Stages 1–2, `dataset` must be the only dataset-specific input. Do not hardcode URLs, filenames, column names, class values, or row counts for your Iris development run. Stage 4 receives no dataset-specific input; Codex selects its three datasets under the rules in Part 4.
- For each dataset, use the same final numeric matrix for every method and every value of q. Do not silently change a dataset between computations or plots.
- Exclude targets, class labels, identifiers, names, timestamps used only as identifiers, and other non-feature metadata. The analysis is unsupervised; labels must not determine the embedding.
- Retain your prompts, relevant AI responses, generated code, manual changes, and verification results. You are responsible for every submitted claim and line of code.
- A final consolidated prompt must contain all requirements discovered during follow-up conversation. The grader should not need to reconstruct your intent from a long chat history.

1 Build the dataset acquisition prompt

Write a prompt for ChatGPT that is followed by a short underscore-separated dataset query. The prompt must direct ChatGPT to search the UCI Machine Learning Repository at `https://archive.ics.uci.edu/`, select the most likely matching dataset, inspect its documentation and files, and create a numeric observation-feature matrix. Use `Iris` for the required development run. This is deliberately a high-level AI task: the prompt must express the goal and checks without assuming the dataset's file layout in advance.

Required behavior

- Use the official UCI repository page and the files linked from it. Record the selected dataset's official title, UCI dataset ID, page URL, the reason it was selected, and the exact source file or files used.
- Interpret underscores in `dataset` as spaces for the repository search. Use all resulting words as search keywords, select the most likely UCI result, and do not ask the user to disambiguate among plausible matches.
- Determine which rows are observations and which columns are candidate features by reading the documentation and inspecting the data.
- Remove target or class columns, identifiers, and metadata. Retain only numeric unsupervised features. If a categorical variable cannot be retained without inventing a representation, exclude it and explain why.
- Recognize documented missing-value markers. State the chosen treatment. Do not silently impute values; if rows or columns are removed, report how many.

- Write `dataset.csv` with one observation per row and one numeric feature per column. Include concise, stable feature names in the header.
- Stop with a clear explanation if the repository cannot be reached or the dataset cannot be converted unambiguously. Do not fabricate data or documentation.

Inspect before transforming. The point of this stage is to let the AI adapt to unfamiliar packaging while making its choices visible. A successful run includes a short audit trail, not only a CSV file.

Your data prompt

Submit `Prompt1.txt`. Execute it by giving ChatGPT one message formed by concatenating the contents of `Prompt1.txt`, one or more whitespace characters (spaces and/or line breaks), and `dataset`, with no other dataset-specific editing. For the required development run, the message consists of the prompt contents followed by whitespace and `Iris`; the resulting file is `Iris.csv`.

2 Produce centered and uncentered PCA

Write `Prompt2.txt` to ask Codex to select established Python library routines for both PCA variants and operate on `dataset.csv`. Execute it by giving Codex one message formed by concatenating the contents of `Prompt2.txt`, one or more whitespace characters (spaces and/or line breaks), and `dataset`. You are not required to implement PCA from first principles. You are required to verify that the selected routines perform exactly the operations defined below.

You may assume that `dataset.csv` already exists. Run the final prompt with the argument `Iris` for the required development output.

Method	Matrix analyzed	Required interpretation
Uncentered PCA	X	Use the three leading right singular directions of the original matrix. Do not subtract column means and do not scale columns.
Centered PCA	$X - \mathbf{1}\mu^{\mathsf{T}}$	Subtract each column mean, then use the three leading principal directions. Do not divide by column standard deviations.

PCA requirements

- Name the library, routine, and relevant parameter settings. Explain how those settings create centered or uncentered PCA.
- Generate the following files:
  `dataset_upca_2D.png` and `dataset_cpca_2D.png`;
  `dataset_upca_3D.png` and `dataset_cpca_3D.png`. The 2D plots use PC1 and PC2; the 3D plots use PC1, PC2, and PC3.
- Use equal axis scaling in every displayed dimension, visible axis labels, a descriptive title, and the same point order in all outputs. Do not rescale or rotate coordinates after projection.

- Do not use class labels to color, annotate, select, or otherwise influence points in the required plots.
- Report the three singular values or eigenvalues used and a simple numerical verification of centering behavior, such as the norm of the projected-data mean for centered PCA.

Expected ambiguity. The sign of each PCA axis is arbitrary. Correct 2D and 3D plots may therefore be reflected along any principal axis. This does not change the point configuration.

Example output on the Iris dataset

Figure **??** shows representative 2D and 3D output obtained from the four numeric Iris features. Class labels are not used. The uncentered panels use the leading right singular directions of X; the centered panels first subtract the column means.

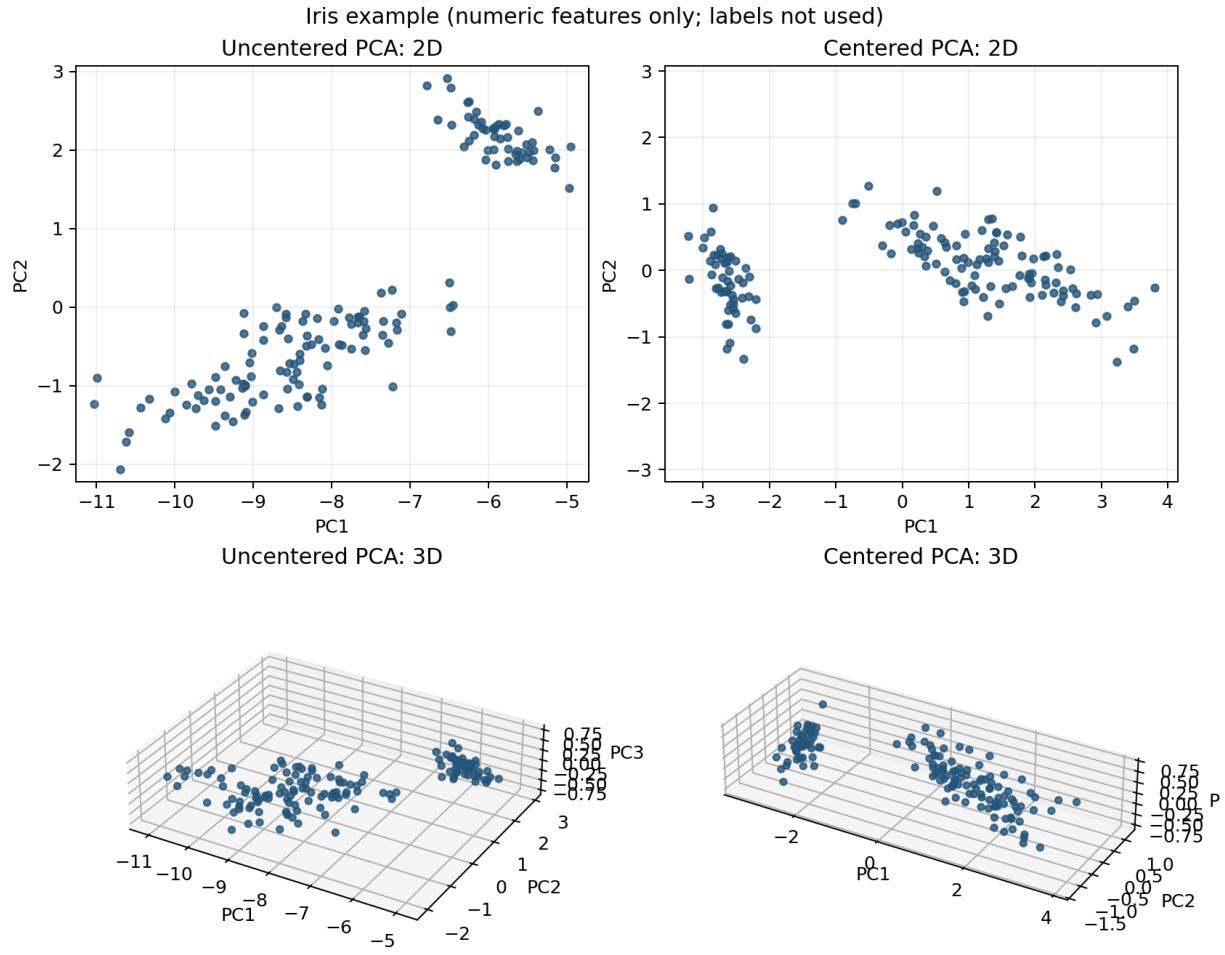


Figure 1: Example 2D and 3D uncentered and centered PCA output for Iris.

3 Implement classical MDS with Codex

Create `Prompt3.txt`. In a fresh Codex task, attach the class handout `MDS.pdf` and then submit the contents of `Prompt3.txt`. The prompt must identify the attached `MDS.pdf` as the mathematical specification and instruct the AI to produce one file, `mds.py`, implementing the STRAIN or classical MDS algorithm derived in the handout. A high-level MDS routine is not permitted. General array, distance, plotting, and symmetric eigendecomposition operations are permitted.

Distance input

For observations x_i and x_j in p dimensions, compute the Minkowski distance with an exponent q selected at run time:

$$d_q(x_i, x_j) = \left(\sum_{\ell=1}^{p} |x_{i\ell} - x_{j\ell}|^q \right)^{1/q}, \qquad q \geq 1. \tag{1}$$

Also support the limiting max-coordinate distance:

$$d_\infty(x_i, x_j) = \max_{1 \leq \ell \leq p} |x_{i\ell} - x_{j\ell}|. \tag{2}$$

Implementation requirements

- Implement pairwise Minkowski distances, the Gram-matrix construction, eigenvalue ordering, coordinate construction, and requested diagnostics explicitly in `mds.py`.
- Do not call a library function that directly performs MDS, classical scaling, kernel PCA, or PCA as a substitute for the handout algorithm.
- Use a symmetric eigensolver. Account for small round-off asymmetry and explain the numerical tolerance used when distinguishing negligible from meaningful negative eigenvalues.
- Use this exact command-line interface:

  ```
  python mds.py --input dataset.csv --q Q --dimensions K
  ```

  Here, `Q` is a real value greater than or equal to 1, or the literal `inf`; `K` is 2 or 3.
- Construct every output filename from the input stem, q, and the requested dimension. The input stem is the input file's base name after removing its directory and final `.csv` extension; preserve its capitalization and underscores. Normalize a finite q to its shortest decimal form without unnecessary trailing zeros, so `2`, `2.0`, and `2.00` all give the tag `q2`. Use the tag `qinf` for $q = \infty$. The two output names are then
  `STEM_mds_qQ_KD.csv` and `STEM_mds_qQ_KD.png`,
  where `STEM` is the input stem, `Q` is the normalized value of q (or `inf`), and `K` is 2 or 3. For example, `Iris.csv` with $q = 2$ and $K = 2$ produces `Iris_mds_q2_2D.csv` and `Iris_mds_q2_2D.png`. With $q = 1.5$ and $K = 3$, the same input produces:
  `Iris_mds_q1.5_3D.csv` and `Iris_mds_q1.5_3D.png`.
- Each run must write a CSV file containing the embedding coordinates and a PNG file containing the visualization. It must also report the full eigenvalue spectrum in a clear machine-readable form on standard output.

- For the development dataset, run the program at $q = 2$ in both dimensions. The resulting visualizations must be named `Iris_mds_q2_2D.png` and `Iris_mds_q2_3D.png`.
- Include automated tests on at least one small configuration whose pairwise distances or embedding can be checked by hand.

Required validation at q = 2

`Prompt4.txt` will use `mds.py` to perform the required validation. For Euclidean distance ($q = 2$), classical MDS applied to each selected matrix X must reproduce its centered PCA point configuration in both two and three dimensions. The coordinates need not have the same orientation: rotation and reflection are allowed. Compare each pair of configurations after optimal orthogonal alignment or compare their pairwise Euclidean distance matrices. Report the numerical discrepancies in `q_analysis.md`.

A plot alone is not the validation. The visual match is useful, but your submission must also include a numerical comparison that is invariant to rotation and reflection.

Example output on the Iris dataset

Figure **??** shows representative 2D and 3D output from classical MDS applied to the Iris feature matrix with $q = 2$. It illustrates output produced by `mds.py`; the required validation remains numerical and is performed in Part 4.

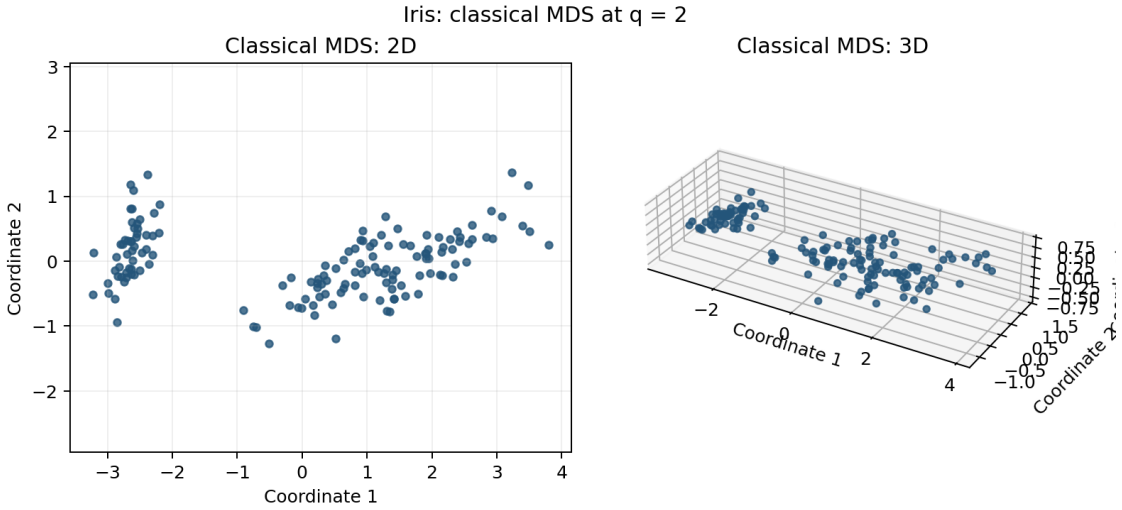


Figure 2: Example 2D and 3D classical MDS output for Iris at $q = 2$.

Prompt3.txt

Submit `Prompt3.txt`, `mds.py`, `Iris_mds_q2_2D.png`, and `Iris_mds_q2_3D.png`. The prompt must identify the attached `MDS.pdf` as the mathematical specification, list the allowed and prohibited library operations, reproduce the exact command-line interface above, require the AI to run its tests, and require any assumption to be stated. The two PNG files must show the $q = 2$ two-

and three-dimensional embeddings of the same development dataset used in Stage 2. The broader numerical and graphical investigation is created in Part 4.

4 Investigate the Minkowski exponent q

Write `Prompt4.txt`. Execute it by giving Codex the contents of that file, without appending a dataset query, with `mds.py` available. The prompt must direct Codex to select exactly three datasets from the UCI Machine Learning Repository, prepare their numeric observation-feature matrices, run the same q experiment on each, and compare the results. Unlike raising all distances to a power, changing q can change which pairs are relatively near or far, so this is a study of changing geometry rather than only distance contrast.

The required deliverable for this part is the analysis in `q_analysis.md`. Numerical results should appear in that analysis, preferably in tables. Images are supporting evidence rather than independent required outputs and should be included only when they help establish or explain a conclusion.

Dataset-selection protocol

The rules in this subsection are requirements that the student's `Prompt4.txt` must impose on Codex.

- Select exactly three datasets from the official UCI Machine Learning Repository. Do not ask the user to supply or approve the dataset names.
- Complete and record the selection before computing any q-dependent results. Dataset selection must not be based on which datasets produce attractive plots or preferred numerical outcomes.
- Apply the acquisition rules from Part 1 to each dataset: use the official UCI page and files, identify observations and candidate features from the documentation, remove targets and non-feature metadata, retain only numeric unsupervised features, and document missing-value treatment.
- Each selected dataset must yield at least three usable numeric feature columns and must be computationally feasible for explicit pairwise distances and classical MDS. State the resulting matrix dimensions and justify feasibility before analysis.
- Select datasets that provide meaningful contrasts in properties potentially relevant to Minkowski geometry, such as numbers of observations and features, feature units or scale imbalance, correlation, sparsity, concentration, or clustering structure. Explain the selection rationale.
- Record each official dataset name, UCI URL, exact source file or files, safe filename slug, matrix dimensions, relevant feature properties, exclusions, and missing-value treatment.
- Do not replace a selected dataset after inspecting its q-dependent results. Replacement is allowed only if the dataset proves technically ineligible under the rules above. Document the reason and choose the replacement before inspecting its q-dependent results.
- Save each final matrix as `data/stage4/dataset_slug.csv`, using the recorded safe slug in place of `dataset_slug`. These matrices are retained for reproducibility but are not separate Stage 4 analysis outputs.

Minimum experiment

- Apply the same experiment and analysis rules to all three datasets.

- For every dataset, include $q = 1$, $q = 2$, at least one value strictly between 1 and 2, at least two finite values greater than 2, and $q = \infty$. Additional values may be chosen adaptively if they reveal a transition; document any dataset-specific additions.
- Report a numerical comparison with one row per dataset and value of q in the analysis. Include at least: number of meaningfully negative eigenvalues, magnitude of the negative eigenvalue mass, proportions of positive eigenvalue mass represented by the first two and first three dimensions, and normalized distance errors for both embeddings.
- When plots help support the analysis, use a consistent point style, panel size, axis convention, and point order across q values for a given dataset. Each panel must identify both the dataset and q.
- For every dataset, include at least one neighborhood or distance-order measurement relative to $q = 2$, such as nearest-neighbor agreement or rank correlation of pairwise distances.
- Ask the AI to identify visible patterns and possible explanations, then check those claims against the numerical results. Separate observations about an individual dataset from patterns shared across the three datasets and from claims that follow mathematically.
- Discuss sensitivity to feature units for every dataset. Because no feature scaling is performed, explain whether each dataset's documented units make comparisons across q meaningful and identify features that appear to dominate distances.
- For every dataset, perform the $q = 2$ comparison with centered PCA required in Part 3.
- Write the dataset-selection record, complete interpretation, numerical results, cross-dataset comparison, and $q = 2$ PCA validations to `q_analysis.md`. Include or reference supporting images only as needed.

Questions your analysis must answer

- For each dataset, what changes as q moves away from 2, and which changes are visible versus numerical?
- Within each dataset, which observations or neighborhoods are most stable across q? Which are least stable?
- For each dataset, at which q values is the Gram matrix farthest from positive semidefinite, and what does that imply about representing those distances in Euclidean space by classical MDS?
- How much of the usable positive spectral mass is captured in two and three dimensions for each dataset and value of q?
- Why is $q = 2$ special in relation to centered PCA?
- Which effects appear consistently across the three datasets, and which appear dataset-specific?
- Which conclusions follow from Minkowski geometry itself, and which are only empirical observations from the selected datasets and their units?

Prompting expectations

A strong prompt specifies the goal, relevant context, output files, constraints, and verification. For Codex, name the behavior to implement, point to the attached handout, state which operations are prohibited, and require tests. You may use follow-up messages while developing, but submit a final self-contained version of each prompt.

Prompt	Must control	Evidence of success
`Prompt1.txt`	UCI source selection, feature/label distinction, missing data	`dataset.csv` from an unfamiliar UCI dataset
`Prompt2.txt`	Exact centered and uncentered definitions, library settings, plot conventions	Four correctly named 2D and 3D PCA visualizations
`Prompt3.txt`	Attached `MDS.pdf`, handout algorithm, permitted operations, exact interface, tests, q support	`mds.py` with the required command-line interface and the two named $q = 2$ visualizations
`Prompt4.txt`	Bounded selection of three datasets, `mds.py` execution, common q experiment, measurements, comparisons, limits on generalization	Comparative `q_analysis.md`, with supporting figures where useful

Final checklist

- ☐ All prompts are self-contained. `Prompt1.txt` and `Prompt2.txt` accept a new UCI dataset query; `Prompt4.txt` receives no dataset-specific input.
- ☐ Every submitted matrix contains numeric features only, with no labels or identifiers.
- ☐ PCA uses the specified centering and scaling rules and produces all four named images.
- ☐ `mds.py` follows `MDS.pdf`, has the exact interface, and produces `Iris_mds_q2_2D.png` and `Iris_mds_q2_3D.png` for the development dataset.
- ☐ Stage 4 selects exactly three eligible UCI datasets before computing q-dependent results and documents the selection and any permitted replacement.
- ☐ The same q experiment is applied to all three selected datasets, and the numerical results, $q = 2$ validations, and comparative interpretation appear in `q_analysis.md`.
- ☐ All prompts, outputs, records, and instructions needed for a clean rerun are present.

Submission structure

Submit one archive with the following top-level structure. Equivalent additional files are allowed, but do not rename the required files. The Stage 4 CSV entry stands for one file per selected dataset; `supporting_images/` is optional.

```
README.md
prompts/
  Prompt1.txt
  Prompt2.txt
  Prompt3.txt
  Prompt4.txt
data/
  Iris.csv
  stage4/
    dataset_slug.csv
code/
  mds.py
results/
  Iris_upca_2D.png
  Iris_cpca_2D.png
  Iris_upca_3D.png
  Iris_cpca_3D.png
  Iris_mds_q2_2D.png
  Iris_mds_q2_3D.png
  q_analysis.md
  supporting_images/
ai_record/
  transcript_or_links.md
  change_log.md
```

README requirements

- State that `Iris` was used for the development run, list the three datasets selected in Stage 4, and provide commands or a short sequence that reproduce every required result from the submitted numeric matrices.
- List required Python packages and versions. Use a fixed random seed wherever a selected routine is randomized.
- Explain where a grader substitutes a different UCI dataset query for Stages 1–2, how those files are generated, and that Stage 4 instead selects its own three datasets.
- Identify every manual code change made after AI generation. Briefly explain why it was necessary.
- Confirm that a clean run does not depend on files outside the submitted archive, except retrieval of the named public UCI datasets.

Hidden-dataset evaluation

The instructor or TA may provide a dataset query not disclosed in advance. They may then run `Prompt1.txt` and `Prompt2.txt`. The hidden dataset will be selected so that a numeric unsupervised matrix and the required 2D and 3D visualizations are reasonable under the stated rules. Stage 4 receives no hidden dataset query; its dataset selection is part of the evaluated task. Evaluation may include inspection of the selection record, point configurations, analysis, and numerical checks.

- Reflections and rotations of a correct configuration are accepted where mathematically appropriate.
- The grader will not reward labels, colors, decorative styling, or manual annotations that reveal the hidden class structure.
- Failure to retrieve a source must produce an honest, useful error message. Fabricated or silently substituted data receives no credit for that run.
- A prompt that succeeds only because it names the development dataset's columns or filenames is considered hardcoded.